AN2DL - First Challenge Report
ReLUctant

Alessandro Del Fatti, Matteo Garzone, Francesco Genovese

Kaggle: alessandrodelfatti, matteogarzone, frncscgnvs

270240, 279709, 307916

November 17, 2025

1 Introduction

This report presents our approach to the AN2DL course first challenge, tackling a supervised deep learning task on the *Pirates Pain* dataset.

The presented problem is a a multiclass time series classification with the objective to predict pain levels experienced by pirates based on the biomechanical sensor data provided.

The task involves classifying pain into three categories: **No Pain** (class 0), **Low Pain** (class 1), and **High Pain** (class 2).

The challenge presents significant difficulties due to severe class imbalance (77% No Pain, 14% Low Pain, 9% High Pain), high-dimensional temporal data, and the need to capture temporal dependencies in the data.

2 Data profiling

Before starting the training process of the model, we decided to perform a first step of data profiling. In particular, we took advantage of the `YDataProfiling` Python library[1]. Using this tool, we noticed that in the dataset:

- Feature `joint_30` is constant. We decided to remove it, as it provides no information.
- `n_legs=1`, `n_hands=1`, and `n_eyes=1` match for each pirate. We combined them into one feature: `has_prosthesis`.
- Many features are **highly correlated** as reported by the heatmap in figure 1. To handle this, we decided to consider the average of the columns that showed a correlation higher than 0.8.

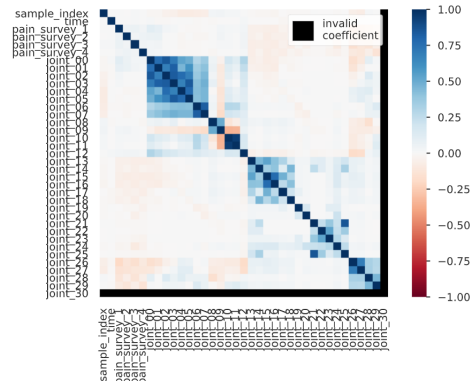

Figure 1: Heatmap of the correlations between features

In addition to the profiling analysis, we computed the autocorrelation function (ACF) for the numerical features. The average **first negative lag** is 29.93, the first lag with $|ACF| < 0.1$ is 4.60, and the **average maximum peak lag** is 1.26. The ACF provided valuable insights into the temporal dependencies of our data and helped improve model

[1] https://docs.profiling.ydata.ai/

performance. However, since the ACF was computed following the logbook's suggestion after an initial, time-consuming grid search for selecting window size and stride, the window size values from the grid search do not fully align with the ACF insights.

3 Method

3.1 Preprocessing Pipeline

Our preprocessing pipeline consists of several critical steps:

1. Normalization: We applied minmax normalization to avoid gradient explosion.

2. Temporal Encoding: Added cyclical time-position encoding features to capture sequence progression. With P set to obtain days and weekdays.

$$x_{\sin} = \sin\left(\frac{2\pi x}{P}\right), \quad x_{\cos} = \cos\left(\frac{2\pi x}{P}\right)$$

3. Sequence Windowing: Applied sliding windows with a `size=10` and `stride=5` to create shorter sequences. This increased the number of samples for the training and validation steps.

3.2 Data Augmentation

To address class imbalance, we applied targeted oversampling of minority classes. We tried two approaches:

- Computation of inverse-proportional class weights for loss function weighting
- Oversampling of the less populated classes

3.3 Training Strategy

3.3.1 Loss Function

Categorical cross-entropy with class weights computed for minority class loss weighting. We also experimented with:

- *Label Smoothing loss:* a custom made categorical cross-entropy extension that also maps class labels to properly-distanced floats rather than equally-distanced integers.
- *Focal loss:* a categorical cross-entropy extension designed to address class imbalance in classification tasks by giving higher weights to hard-to-classify data samples while down-weighting samples classified with higher confidence.

3.3.2 Regularization Techniques:

We performed a grid search in four steps, obtaining the best performance with the configurations in table 1. For complete reference, consult the training logs in the notebooks. We noticed that, to achieve better performance in terms of F1-score, **a small window size** (as discussed before) **and low values for the regularization terms ℓ_1 and ℓ_2 are preferable**. So, starting from those values, we tried to further reduce them, obtaining better results:

- ℓ_1 regularization: best if turned off.
- ℓ_2 regularization: $1e-5$
- Dropout: 0.2 @ CNN-1d, 0.3 @ LSTM

We also experimented with:

- Label smoothing: $\alpha = 0.1$
- Gradient clipping and normalization: $max_{|\nabla L|} = 1.0$

3.3.3 Optimization:

We used `AdamW` as optimizer. We set the learning rate and the batch size according to the results of the grid search and we used early-stopping with `patience=50`. As LR scheduler, we used `ReduceLROnPlateau(factor=0.5, patience=40)`

4 Experiments

As a first approach, we tried using baseline models: RNN, LSTM, GRU and their relative bidirectional versions. By evaluating them, we noticed that the **bidirectional LSTM** performs better consistently, across different configurations of hyperparameters.

	RNN	BiRNN	LSTM	BiLSTM	GRU	BiGRU
F1	0.6294	0.7941	0.6294	**0.8562**	0.6294	0.8030

After choosing a model, we used grid search to find a good set of hyperparameters for the network.

Since our model was lacking accuracy on the less populated class (**High pain**), we tried to address the problem by implementing sample augmentation and using loss functions that scale up errors in minority classes.

Lastly, adding a 1D Convolutional layer and an Attention layer to the network, resulted in improved performance.

We experimented with a **hierarchical classification** approach: we used a binary classifier to discriminate **No Pain** from **Pain** labeled samples and

a second binary classifier to distinguish the levels of pain **High** and **Low** of samples classified as painful. This approach proved to be overfitting for the hyperparameter set we used, but sure is a promising path to be explored for such a task.

5 Results

We experimented with different model configurations, gradually adding modules and features.

As table 2 suggests, a bidirectional LSTM based classifier with a convolutional and an attention layer, which takes into account temporal features as well, trained with gradient clipping, LR scheduling, and minority classes oversampling, is the best performing.

6 Discussion

6.1 Strengths of Our Approach

1. **Code modularity:** Having our project set up in modules that load configuration files for training, using the notebook only as a training pipeline helped us to add functionality, debug, and compare approaches easily.
2. **Robustness:** Consistent performance despite severe class imbalance.
3. **Proper Model Selection:** Chose a simpler model over a higher validation score.

6.2 Limitations and Lessons Learned

1. **Augmentation Risk:** Aggressive augmentation can create unrealistic patterns that do not generalize.
2. **Hierarchical Overfitting:** High validation scores do not guarantee test performance.
3. **Data analysis:** the usefulness of ACF computation in time series analysis is evident. In retrospect, conducting a thorough dataset profiling before performing the grid search would likely have been more beneficial.

7 Conclusions

This work addressed the task through a comprehensive approach, combining data profiling with feature and NN architecture engineering. Our final model achieved a test weighted F1-score of 0.9616 on the provided open public test set, demonstrating strong generalization despite severe class imbalance.

Key contributions include systematic feature engineering guided by profiling, correlation analysis, and ACF computation; effective handling of class imbalance through targeted oversampling and weighted loss functions; a modular code architecture that facilitated rapid experimentation and hyperparameter optimization.

Future work could explore advanced augmentation techniques specifically designed for biomechanical time series, ensemble methods combining multiple architectures, and further refinement of the hierarchical classification approach to reduce overfitting while maintaining its validation performance advantage.

Window	**Stride**	**Batch**	**LR**	**Hid. Layers**	**Hid. Size**	ℓ_1	ℓ_2	**Dropout**	**F1**
50	25	64	3e-4	1	256	1e-4	1e-4	0.4	0.9168 ± 0.0371

Table 1: Grid search: consolidated best configuration from all the 4 steps

Model	**Val F1**	**Test F1**	**Params**	**Notes**
Baseline Bi-LSTM (1hl, 256h)	0.9168	0.9051	392K	Grid search hyperparameters
Bi-LSTM + LRScheduling + L2 reg	0.9232	0.9163	392K	Better regularization and convergence
Bi-LSTM + CNN1d	0.9288	0.9142	855K	Added CNN 1d layer
Bi-LSTM + Temp + CNN1d	0.9305	0.9208	855K	Temporal features added
Bi-LSTM + Temp + CNN1d + Attention	**0.9312**	**0.9616**	**878K**	Added an attention layer
Hierarchical 2-Step classifier	0.9925	0.9293	856K	Overfitting

Table 2: Performance comparison of different model configurations. Best results in bold